

Vision-Based Predecessor-Following Warehouse Convoys

Project Handbook, Simulation User Guide,
and Evidence Reference

Neeraj Kumar Kanchani

EE 616-57, Section 9189, Four Credits
Faculty Reviewer: Professor Masudul Imtiaz

Frozen Gate 7 edition
September 30, 2026

Document purpose

This handbook explains the complete software and simulation project in a form that can be studied, operated, changed, and defended. It is not the final faculty report. The faculty report remains the project owner's own writing. The handbook is a technical reference for learning the system, reproducing the experiments, understanding every recorded metric, and planning later revisions.

The implemented system is a ROS 2 Jazzy and Gazebo Harmonic simulation of one route-owning leader and independently driven predecessor-following robots. Every follower uses a forward RGB camera and local wheel odometry. The accepted pipeline contains YOLOv8n target detection, monocular range and bearing, a relative-state extended Kalman filter, bounded formation control, and the states `TRACK`, `PREDICT`, and `SAFE_STOP`. Gazebo pose truth is restricted to evaluation.

This document deliberately distinguishes four levels of evidence:

1. mathematical definitions and software design;
2. deterministic toy-model results;
3. ROS 2 and Gazebo simulation results on one laptop; and
4. physical or industrial claims, which have not been demonstrated.

Assistance and responsibility statement

An AI assistant supported code development, experiment organization, documentation structure, editing, and generation of this handbook. Neeraj Kumar Kanchani is responsible for reviewing the source, verifying the evidence, understanding the equations and commands, and deciding what to submit or present. This handbook must not be used to conceal AI assistance or to claim work that the project owner cannot explain.

How to use this book

Read Part I before explaining the project to another person. Read Part II when studying the algorithms. Use Part III while operating the software. Use Part IV before changing parameters or designing a new experiment. Use Part V when interpreting results. The appendices provide command, topic, parameter, definition, and oral-examination references.

Commands assume that the current directory is the single authorized workspace:

```
/home/justneeraj/Documents/MS_sem3/GRAD_PROJECT
```

Do not create a second project workspace. Generated ROS directories remain inside `work/ros2_ws/`.

Contents

Document purpose	i
Assistance and responsibility statement	ii
How to use this book	iii
I Problem, Scope, and Engineering Foundations	1
1 The core problem	2
1.1 Application problem	2
1.2 Why the problem is difficult	2
1.3 Implemented solution	2
1.4 Why this project is worth doing	3
2 System boundary and claims	4
2.1 Inside the research system	4
2.2 Outside the research system	4
2.3 Ground-truth firewall	4
2.4 Safe language	5
3 Essential robotics definitions	6
3.1 Pose, frame, and transform	6
3.2 Range and bearing	6
3.3 Odometry	6
3.4 Open loop and closed loop	6
3.5 Determinism and repeatability	6
II Implemented Architecture and Algorithms	7
4 Robot topology and ROS namespaces	8
4.1 Chain structure	8
4.2 Namespace rule	8
4.3 Package responsibilities	8

5	Gazebo robot and sensor model	10
5.1	Robot body	10
5.2	Camera	10
5.3	Rear visual target	10
5.4	Physics and rendering	10
6	Visual perception	11
6.1	Why use a known target	11
6.2	YOLOv8n input contract	11
6.3	Detection selection	12
6.4	Why the failed detectors matter	12
7	Monocular range and bearing geometry	13
7.1	Focal length from horizontal field of view	13
7.2	Bearing	13
7.3	Range	13
7.4	Failure mechanisms	14
8	Relative-state extended Kalman filter	15
8.1	State vector	15
8.2	Prediction model	15
8.3	Measurement update	15
8.4	Initialization	16
8.5	Interpretation limit	16
9	Bounded control and visual-loss supervision	17
9.1	Control law	17
9.2	Supervisor states	17
9.3	What SAFE_STOP means	17
10	Leader routes and disturbances	19
10.1	Deterministic leader	19
10.2	Nominal profiles	19
10.3	Controlled disturbances	19
III	Environment, Setup, and Daily Operation	20
11	Canonical software environment	21
11.1	Why the project uses Docker	21
11.2	Target laptop	21
11.3	Container services	21
12	First-time setup	22
12.1	Open the only authorized workspace	22
12.2	Build and validate the container	22
12.3	Open VS Code	22
12.4	Verify the shell	22

13 Daily development workflow	24
13.1 Recommended sequence	24
13.2 ROS inspection commands	24
13.3 Stopping processes	24
14 Using Gazebo interactively	25
14.1 Open the playground	25
14.2 GUI operations	25
14.3 Controlled scenario templates	25
14.4 AWS warehouse	25
15 Running the frozen experiments	26
15.1 Important overwrite warning	26
15.2 Environment gate	26
15.3 Camera measurement gates	26
15.4 Nominal pair	26
15.5 Incremental chain	26
15.6 Controlled disturbances	27
15.7 Timing and resources	27
15.8 Run one launch manually	27
IV Configuration and Safe Modification	28
16 Configuration hierarchy	29
16.1 Where values live	29
16.2 Precedence	29
17 Changing a scenario	30
17.1 Static scenario schema	30
17.2 Pose fields	30
17.3 Safe scenario-change procedure	30
18 Changing perception parameters	32
18.1 Confidence	32
18.2 IoU threshold	32
18.3 Inference size	32
18.4 Device	32
18.5 Range calibration	32
19 Changing control parameters	33
19.1 Desired gap	33
19.2 Gains	33
19.3 Timeouts	33
19.4 Bounds	33
19.5 Control rate	33

20	Creating a new experiment correctly	34
20.1	Do not edit frozen evidence	34
20.2	Minimum protocol	34
20.3	One-variable example	34
20.4	When optimization is justified	35
V	Metrics, Evidence, and Interpretation	36
21	Measurement metrics	37
21.1	Range RMSE	37
21.2	Bearing MAE	37
21.3	Precision and recall	37
21.4	Valid measurement rate	37
22	Formation and recovery metrics	38
22.1	Spacing RMSE	38
22.2	TRACK fraction	38
22.3	Reacquisition time	38
22.4	Collision sample	38
22.5	Rearward error propagation	38
23	Timing and resource metrics	39
23.1	Latency stages	39
23.2	Percentiles	39
23.3	Real-time factor	39
23.4	CPU interpretation	39
24	Validation gates and frozen results	40
24.1	Gate 1: environment	40
24.2	Gate 2: camera measurement	40
24.3	Gate 3: one pair	40
24.4	Gate 4: incremental chain	40
24.5	Gate 5: controlled disturbances	40
24.6	Gate 6: target-laptop timing	41
24.7	Gate 7: freeze	41
25	Limitations and threats to validity	42
25.1	Simulation-to-reality gap	42
25.2	Target assumption	42
25.3	Occlusion model	42
25.4	Actuator model	42
25.5	Chain length and routes	42
25.6	Timing instrumentation	42
25.7	Known implementation debt	43

VI Troubleshooting and Extension	44
26 Troubleshooting	45
26.1 Blank host terminal or missing ROS command	45
26.2 Docker permission denied	45
26.3 Gazebo GUI does not open	45
26.4 CUDA or GPU not visible	45
26.5 No camera frames	45
26.6 No detections	45
26.7 Immediate <code>SAFE_STOP</code>	46
26.8 A gate script fails	46
27 Reasonable future directions	47
27.1 Resource-aware perception	47
27.2 Physical occlusion scenes	47
27.3 Leader autonomy	47
27.4 Robot assignment	47
27.5 Physical robot validation	47
A Command reference	48
B Topic and message reference	49
C Accepted control parameters	50
D Glossary	51
E Oral defense study questions	53
F Frozen evidence locations	54
Index	55

List of Tables

List of Figures

2.1	Runtime information boundary. Dashed arrows are logging paths.	5
4.1	Implemented maximum convoy used in the accepted demonstration.	8
9.1	Implemented measurement-age supervisor.	18

Part I

Problem, Scope, and Engineering Foundations

Chapter 1

The core problem

1.1 Application problem

The application is warehouse material transport. A leader owns a route. One or more independently driven carts follow it. The carts are not mechanically connected. Follower 1 observes the leader, Follower 2 observes Follower 1, and each later follower observes its immediate predecessor. This is a **predecessor-following topology**.

The technical problem is not simply “make robots follow.” The measurable question is:

How accurately and reliably can a resource-constrained follower maintain a requested predecessor-relative gap using a forward RGB camera and local wheel odometry during changes in range, bearing, route curvature, target visibility, and bounded disturbances?

The follower does not receive the leader’s global route, global pose, or Gazebo pose. It must infer relative geometry from the image and combine that measurement with its own motion estimate.

1.2 Why the problem is difficult

Five coupled effects make the problem interesting.

1. A monocular image does not directly contain metric depth. Metric range requires known geometry or learned depth cues.
2. A detector can miss, clip, or mislocalize a target. Image errors become range and bearing errors.
3. A controller changes the camera viewpoint, so perception and motion form a closed loop.
4. Temporary visual loss requires a defined recovery policy. Continuing indefinitely is unsafe; stopping immediately for every missed frame can be impractical.
5. In a chain, the motion error of an early follower changes the target motion seen by the next follower. Error can grow, decrease, or change with the scenario.

1.3 Implemented solution

The implemented follower executes this sequence:

1. receive a 640 by 480 RGB image;
2. detect a known red rear target with a trained YOLOv8n model;
3. convert its bounding box to relative range and bearing;
4. combine the measurement with local wheel odometry in a four-state EKF;
5. calculate bounded linear and angular commands;
6. select TRACK, PREDICT, or SAFE_STOP from measurement age;
7. publish commands and synchronized diagnostic topics; and
8. log performance using a separate evaluation-only process.

1.4 Why this project is worth doing

The project does not claim invention of leader–follower control, camera following, visual markers, monocular range estimation, or Kalman filtering. Related systems already exist [1, 2, 3, 4, 5, 6]. The defensible contribution is the implementation and evaluation of one constrained architecture:

- warehouse-focused predecessor following;
- independently driven followers;
- forward RGB and local odometry at each follower;
- explicit visual-loss states;
- evaluation-only ground truth;
- pair-first and incremental-chain validation;
- rearward error measurements;
- controlled visual-loss and command-bias tests; and
- timing and resource measurements on a 4 GiB laptop GPU.

This makes the project useful as a systems-integration and experimental-method study. Its strength is not a first-ever algorithm. Its strength is traceable engineering evidence and clear limits.

Chapter 2

System boundary and claims

2.1 Inside the research system

The research system includes leader route generation, follower camera sensing, predecessor detection, relative geometry, estimation, formation control, supervision, ROS 2 communication, Gazebo dynamics, logging, and offline evaluation.

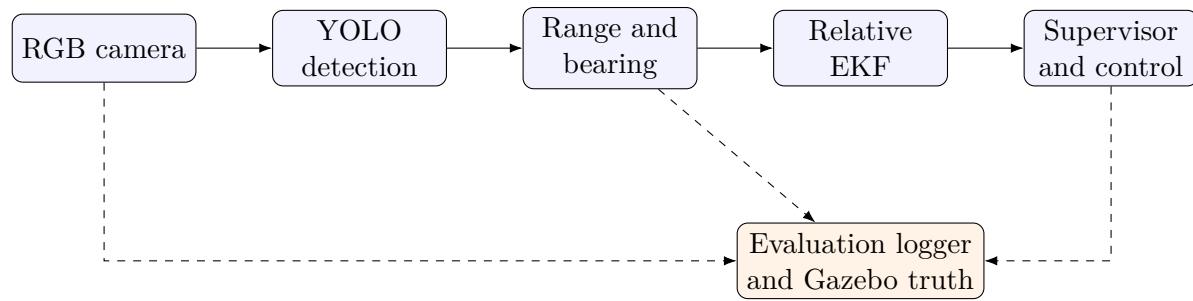
2.2 Outside the research system

The following items are outside the demonstrated scope:

- physical robots;
- safety-rated LiDAR or person detection;
- certified emergency stopping;
- warehouse fleet management;
- wireless network delay and packet loss;
- physical wheel slip and detailed motor dynamics;
- industrial commissioning and standards compliance; and
- production readiness.

2.3 Ground-truth firewall

Ground truth is the simulator’s direct knowledge of model pose. It is useful for calculating error but unavailable to a camera-only physical follower. Therefore, Gazebo pose truth may be consumed only by `ee616_evaluation`. It must never enter `ee616_perception`, the EKF, the supervisor, the controller, or the disturbance proxy.



Truth terminates here; no arrow returns to the robot pipeline.

Figure 2.1: Runtime information boundary. Dashed arrows are logging paths.

2.4 Safe language

Use statements such as “the evaluator recorded zero collision samples under the defined simulation check.” Do not replace that statement with “the system is collision-free” or “the robots are safe.” A finite simulator run cannot establish general safety.

Chapter 3

Essential robotics definitions

3.1 Pose, frame, and transform

A **pose** contains position and orientation. In the planar experiments, a pose is (x, y, ψ) , where ψ is yaw. A **coordinate frame** defines the origin and axes used to express a quantity. A **transform** converts a point or pose between frames.

The follower camera frame is attached to the follower. Relative range and bearing are expressed from that camera viewpoint. Odometry is local motion information published in the robot's odometry frame. Gazebo's world frame is evaluation-only.

3.2 Range and bearing

Range is the straight-line planar distance from camera to target. **Bearing** is the horizontal target angle relative to the camera's forward axis. In this implementation, a target left of image center produces positive bearing because the calculation uses $(c_x - u)/f$.

3.3 Odometry

Wheel odometry estimates motion from wheel movement. The controller uses local forward speed and yaw rate from the ROS `nav_msgs/Odometry` message. Simulation odometry is cleaner than physical wheel odometry; the current evidence does not include real encoder error or slip.

3.4 Open loop and closed loop

An **open-loop** measurement test moves or places a target but does not let the detector control the robot. A **closed-loop** experiment feeds the measurement through estimation and control so that output motion changes later measurements. Gate 2 is measurement-only. Gates 3–6 are closed loop.

3.5 Determinism and repeatability

A deterministic configuration should produce equivalent behavior when initial conditions, software, and hardware scheduling are controlled. Exact floating-point samples can still vary because ROS callbacks and GPU execution are scheduled concurrently. The project therefore uses repeated trials and bounded acceptance criteria rather than requiring byte-identical dynamic CSV files.

Part II

**Implemented Architecture and
Algorithms**

Chapter 4

Robot topology and ROS namespaces

4.1 Chain structure

The leader owns the route. It publishes its own motion command. Each follower runs a copy of the same perception and control stack in an isolated namespace.

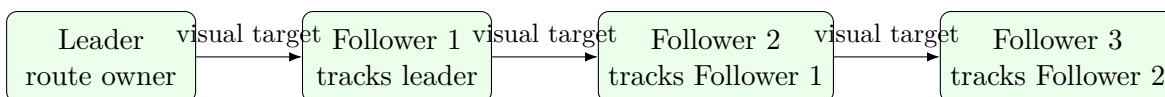


Figure 4.1: Implemented maximum convoy used in the accepted demonstration.

4.2 Namespace rule

Namespaces prevent similarly named topics from colliding. The accepted runtime uses `/leader`, `/follower_1`, `/follower_2`, and `/follower_3`. For example, `/follower_2/odom` is not the same topic as `/follower_3/odom`.

4.3 Package responsibilities

Package	Implemented responsibility
<code>ee616_bringup</code>	Namespace and camera transport probes for environment validation.
<code>ee616_simulation</code>	Gazebo worlds, generated robots, scenario templates, bridges, and disturbance proxy.
<code>ee616_perception</code>	Deterministic red-target baseline, YOLOv8n target detection, range/bearing conversion, and inference diagnostics.
<code>ee616_control</code>	Relative EKF, measurement-age supervisor, bounded follower controller, and deterministic leader route.
<code>ee616_evaluation</code>	Measurement, pair, chain, disturbance, timing, plotting, and summary tools. It alone may read Gazebo truth.

Estimation and supervision are implemented inside `ee616_control`; there are not separate `ee616_estimation` and `ee616_supervisor` packages in the frozen system.

Chapter 5

Gazebo robot and sensor model

5.1 Robot body

Each generated robot has a rectangular base, two driven wheels, and two low-friction caster collision shapes. The body mass is 8.0 kg, each wheel mass is 0.5 kg, wheel radius is 0.10 m, and wheel separation is 0.46 m. Gazebo's differential-drive system consumes a namespaced `cmd_vel` topic and publishes odometry at 30 Hz.

These values form a simplified simulation model. They are not measurements from a selected physical cart.

5.2 Camera

Each follower has a front camera mounted 0.28 m forward and 0.43 m above the base-link origin. It requests 640 by 480 RGB images at 15 Hz, with a 70-degree horizontal field of view and 0.1–30 m clipping range.

5.3 Rear visual target

Every robot that can be observed from behind carries a red rectangular target. The target geometry is 0.08 m thick, 0.70 m wide, and 0.90 m high. The range equation assumes the 0.90 m height is known.

5.4 Physics and rendering

The generated pair and chain worlds use a 0.005 s maximum physics step and a requested real-time factor of 1.0. Shadows are disabled. Recorded experiments are headless. The system reports EGL and rendering fallback warnings on the laptop, so GPU passthrough is verified but accelerated Gazebo rendering is not claimed.

Chapter 6

Visual perception

6.1 Why use a known target

A known rear target makes the first metric range calculation identifiable. The project can separate detection error from the geometry equation. This is a valid engineering baseline, but it assumes the predecessor has a visible target of known dimensions.

6.2 YOLOv8n input contract

The ROS node converts `sensor_msgs/Image` to an RGB NumPy array. Ultralytics' array interface is supplied a contiguous BGR array, so the implementation reverses the color channels before inference. An earlier RGB/BGR mismatch was one reason a prior detector failed.

The accepted v3 configuration is:

Parameter	Value	Meaning
<code>weights_path</code>	v3 checkpoint	Accepted trained model.
<code>class_id</code>	0	The only class is <code>predecessor_target</code> .
<code>confidence</code>	0.40	Validation-selected minimum box confidence.
<code>iou</code>	0.45	Non-maximum-suppression overlap setting.
<code>image_size</code>	640	YOLO inference size.
<code>device</code>	0	First CUDA device.
<code>range_scale</code>	0.9903208	Validation-only linear calibration scale.
<code>range_offset_m</code>	0.1042198	Validation-only linear calibration offset in metres.

6.3 Detection selection

The detector filters predictions to class 0 and selects the highest-confidence remaining box. If no valid box exists, it publishes an invalid measurement using NaN range and bearing and a validity field of zero. A valid measurement has vector fields $(r, \beta, 1)$.

6.4 Why the failed detectors matter

The v1 detector reached only 79.6% valid full-visible measurements and 0.505 m range RMSE. The v2 corrective detector produced zero valid detections on its unchanged final scenes. Both remain recorded failures. The synchronized v3 dataset corrected the image contract and stale-frame label problem and passed a new sealed two-scene gate. Preserving this sequence demonstrates diagnosis rather than selective reporting.

Chapter 7

Monocular range and bearing geometry

7.1 Focal length from horizontal field of view

For image width W and horizontal field of view ϕ , the focal length in pixels is

$$f_x = \frac{W}{2 \tan(\phi/2)}. \quad (7.1)$$

The project uses $W = 640$ and $\phi = 1.2217304764$ rad, which is 70 degrees.

7.2 Bearing

Let u be the detected box center and $c_x = (W - 1)/2$ the principal point. The implementation calculates

$$\hat{\beta} = \tan^{-1} \left(\frac{c_x - u}{f_x} \right). \quad (7.2)$$

This sign convention treats image-left as positive bearing.

7.3 Range

Let H be known target height and h detected pixel height. The forward optical depth is

$$\hat{z} = \frac{f_x H}{h}. \quad (7.3)$$

Because the project reports planar line-of-sight range rather than optical-axis depth, it calculates

$$\hat{r}_{\text{raw}} = \frac{\hat{z}}{\cos \hat{\beta}}. \quad (7.4)$$

The accepted calibration is

$$\hat{r} = 0.9903207646 \hat{r}_{\text{raw}} + 0.1042197665. \quad (7.5)$$

7.4 Failure mechanisms

Range error grows when the target is small, clipped, tilted, partly hidden, or localized with the wrong vertical bounds. Calibration cannot repair a completely missed or incorrect detection. It also must not be fit using final test scenes.

Chapter 8

Relative-state extended Kalman filter

8.1 State vector

The implemented EKF state is

$$\mathbf{x} = \begin{bmatrix} r & \beta & \dot{r} & \dot{\beta} \end{bmatrix}^T. \quad (8.1)$$

It represents relative range, relative bearing, and two predecessor-driven rate terms. It is not a global pose filter.

8.2 Prediction model

Let the follower's measured forward speed be v_f , yaw rate be ω_f , and step be Δt . The nonlinear prediction is

$$r^+ = r + (\dot{r} - v_f \cos \beta) \Delta t, \quad (8.2)$$

$$\beta^+ = \text{wrap} \left[\beta + \left(\dot{\beta} + \frac{v_f \sin \beta}{r} - \omega_f \right) \Delta t \right], \quad (8.3)$$

$$\dot{r}^+ = \dot{r}, \quad (8.4)$$

$$\dot{\beta}^+ = \dot{\beta}. \quad (8.5)$$

The implementation clamps Δt to at most 0.20 s and range to at least 0.05 m. It propagates covariance with the Jacobian $\mathbf{F}$ and additive process covariance $\mathbf{Q}$.

8.3 Measurement update

The measurement is

$$\mathbf{z} = \begin{bmatrix} r_m & \beta_m \end{bmatrix}^T, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}. \quad (8.6)$$

The bearing innovation is angle-wrapped. The measurement covariance uses standard deviations of 0.04 m and 0.5 degrees. The Kalman gain is

$$\mathbf{K} = \mathbf{P}^- \mathbf{H}^T (\mathbf{H} \mathbf{P}^- \mathbf{H}^T + \mathbf{R})^{-1}, \quad (8.7)$$

The source code uses this same matrix expression.

8.4 Initialization

The first valid measurement initializes range and bearing. Both rate terms start at zero. Before initialization, the supervisor commands `SAFE_STOP`.

8.5 Interpretation limit

The filter is an engineering relative-state estimator. It has not been proven observable for all routes, and its covariance consistency has not been tested with normalized innovation statistics. Physical odometry drift and wheel slip remain unverified.

Chapter 9

Bounded control and visual-loss supervision

9.1 Control law

The requested range is $r_d = 1.50$ m. The unsaturated commands are

$$v_c = K_{\dot{r}}\dot{r} + K_r(r - r_d), \quad (9.1)$$

$$\omega_c = K_{\beta}\beta + K_{\dot{\beta}}\dot{\beta}. \quad (9.2)$$

The accepted gains are $K_r = 0.85$, $K_{\dot{r}} = 0.85$, $K_{\beta} = 1.80$, and $K_{\dot{\beta}} = 0.20$. Commands are bounded as

$$v = \text{clip}(v_c, 0, 0.60) \text{ m/s}, \quad (9.3)$$

$$\omega = \text{clip}(\omega_c, -0.80, 0.80) \text{ rad/s}. \quad (9.4)$$

The lower linear bound is zero. Therefore, the follower does not reverse to correct a short gap. This explains the roughly 0.12 m short-gap undershoot retained in stopped-leader pair tests.

9.2 Supervisor states

Let a be the age of the most recent valid measurement.

State	Condition	Action
TRACK	$a \leq 0.20$ s	Use EKF state and full bounded controller.
PREDICT	$0.20 < a \leq 0.75$ s	Use EKF prediction and multiply linear speed by 0.55.
SAFE_STOP	$a > 0.75$ s or filter uninitialized	Publish zero linear and angular command.

9.3 What SAFE_STOP means

It means the software publishes zero commanded velocity after prolonged visual loss. It does not prove a physical stopping distance, emergency-stop integrity, obstacle detection, or functional safety.

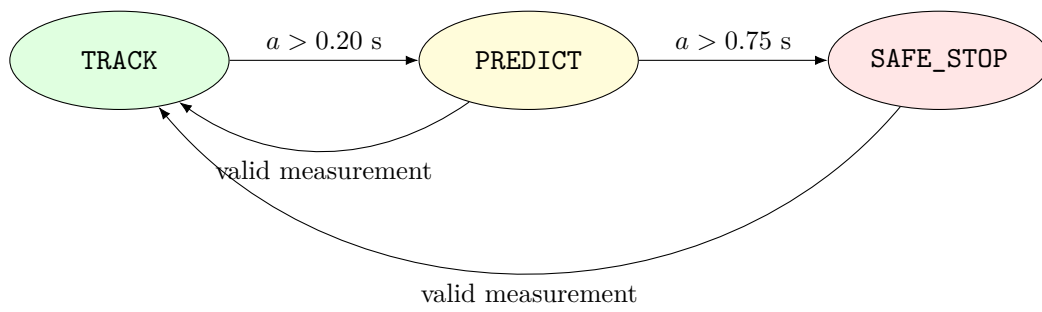


Figure 9.1: Implemented measurement-age supervisor.

Chapter 10

Leader routes and disturbances

10.1 Deterministic leader

The leader is a controlled experimental input. It is not an autonomous warehouse navigator in the frozen gates. Its ordinary linear command is 0.35 m/s. The gradual turn uses 0.12 rad/s; the sharp turn uses 0.18 rad/s.

10.2 Nominal profiles

The **straight** profile moves for 16 s after settling. The **turn** profile moves straight for 5 s, turns for 8 s, and returns to straight motion for 5 s.

10.3 Controlled disturbances

Gate 5 uses five pair scenarios and one combined chain scenario:

- sharp turn;
- 0.5 s full-frame RGB blackout;
- 1.3 s full-frame RGB blackout;
- four-second leader stop;
- 0.5 s post-controller angular-command bias of 0.15 rad/s; and
- a combined route applying the events to Follower 2 in a three-follower chain.

The image blackout replaces all image bytes with zero. It is not a physical object moving between robots. The actuator bias changes the command after the controller and clamps it to the normal limits. It is not wheel slip.

Part III

**Environment, Setup, and Daily
Operation**

Chapter 11

Canonical software environment

11.1 Why the project uses Docker

The laptop host runs Ubuntu 26.04.1 and contains only a partial ROS 2 Lyrical installation. The project does not use that host installation. The canonical container is based on Ubuntu 24.04 Noble with ROS 2 Jazzy, Gazebo Harmonic, `ros_gz`, Python, PyTorch, and Ultralytics.

Docker reduces host-version coupling. It does not make an experiment automatically reproducible; configurations, seeds, weights, versions, and outputs still must be recorded.

11.2 Target laptop

The target is an MSI Katana GF66 12UD with an Intel Core i5-12450H, 16 GB RAM, and NVIDIA RTX 3050 Ti Laptop GPU with 4 GB VRAM. The system must be designed for that resource limit.

11.3 Container services

Service	Purpose
<code>dev</code>	Interactive CPU/software-rendering development shell.
<code>check</code>	Automated environment validation.
<code>gpu</code>	GPU-enabled shell and experiment runner with NVIDIA compute, graphics, and utility capabilities.

The project uses host networking, shared IPC, a 1 GB shared-memory allocation, ROS domain ID 57, Fast DDS, and Gazebo partition `ee616`.

Chapter 12

First-time setup

12.1 Open the only authorized workspace

```
cd /home/justneeraj/Documents/MS_sem3/GRAD_PROJECT
git status --short --branch
```

Do not initialize another repository or copy the project to a second development folder.

12.2 Build and validate the container

```
make container-config
make container-build
make environment-check
make environment-check-gpu
```

The final command requires the NVIDIA Container Toolkit. Docker group membership is root-equivalent access; it is a host security decision, not a project convenience to apply silently.

12.3 Open VS Code

Start VS Code from the logged-in graphical desktop terminal:

```
cd /home/justneeraj/Documents/MS_sem3/GRAD_PROJECT
code .
```

Choose **Dev Containers: Reopen in Container**. The container opens `/workspace`, which is a bind mount of the same authorized host directory, not a second project copy.

12.4 Verify the shell

Inside the container:

```
echo "$ROS_DISTRO"
ros2 --help
gz sim --version
cd /workspace/work/ros2_ws
colcon build --symlink-install
```

```
source install/setup.bash  
ros2 pkg list | grep ee616
```

Chapter 13

Daily development workflow

13.1 Recommended sequence

1. Check Git status and preserve unrelated changes.
2. Open the Dev Container.
3. Run **EE616: Build ROS workspace**.
4. Run **EE616: Test ROS workspace**.
5. Launch one selected scenario or open the learning shell.
6. Inspect topics and nodes before changing code.
7. Stop the simulation cleanly.
8. Record what changed, why it changed, and which evidence was regenerated.

13.2 ROS inspection commands

```
ros2 node list
ros2 topic list
ros2 topic info /follower_1/camera/image
ros2 topic hz /follower_1/camera/image
ros2 topic echo --once /follower_1/control/state
ros2 node info /follower_1/follower_controller
gz topic -l
```

A listed topic proves only discovery. Use `topicinfo`, `topichz`, message inspection, timestamps, and an experiment-specific validator to evaluate data quality.

13.3 Stopping processes

Use Ctrl+C in the active launch terminal. If the terminal was closed, use the VS Code stop task or:

```
bash infrastructure/dev/stop_simulation.sh
```

Chapter 14

Using Gazebo interactively

14.1 Open the playground

From the host desktop session:

```
make gazebo-gui
```

The playground contains a floor, six shelf blocks, a fixed camera, and a visual target. It is for learning. It is not a closed-loop performance experiment.

14.2 GUI operations

Select an entity in the Entity Tree, inspect components, use translation and rotation controls, pause simulation before precise edits, and display collision geometry when studying contact shapes. A GUI-only change is not reproducible until it is saved as source configuration.

14.3 Controlled scenario templates

Run a template with:

```
make scenario-gui SCENARIO=straight_aisle
make scenario-gui SCENARIO=partial_occlusion
make scenario-validate
```

The five development templates are camera calibration, straight aisle, right-angle turn, narrow aisle, and partial occlusion. They share a deterministic seed, camera interface, target geometry, and base lighting.

14.4 AWS warehouse

The pinned AWS no-roof world is an optional visual and resource stress test:

```
make aws-warehouse-gui
make aws-benchmark
```

It does not contain the accepted follower-control experiment and must not be used as evidence of control accuracy.

Chapter 15

Running the frozen experiments

15.1 Important overwrite warning

The experiment runners write into fixed result directories. Running them again can replace frozen local evidence. Verify the Gate 7 package first, and create a newly named experiment protocol and result directory for exploratory changes.

15.2 Environment gate

```
make environment-check-gpu
```

This builds and tests packages, checks namespaces, transports camera data, and records machine information. It is not a follower-performance gate.

15.3 Camera measurement gates

The deterministic camera measurement runner is:

```
docker compose --profile gpu run --rm gpu \  
  bash /workspace/work/ros2_ws/scripts/run_camera_measurement_gate.sh
```

The accepted learned detector runner is:

```
docker compose --profile gpu run --rm gpu \  
  bash /workspace/work/ros2_ws/scripts/run_yolo_measurement_gate_v3.sh
```

15.4 Nominal pair

```
make pair-gate
```

This runs three straight and three gradual-turn trials.

15.5 Incremental chain

```
make chain-gate
```

The script requires all two-follower trials to pass before launching three followers.

15.6 Controlled disturbances

```
make disturbance-gate
```

The script requires all fifteen pair trials to pass before three combined chain trials.

15.7 Timing and resources

```
make timing-gate
```

This repeats the accepted combined scenario three times and adds an observer-only timing process plus 0.25 s resource sampling.

15.8 Run one launch manually

Inside makegpu-shell:

```
source /opt/ros/jazzy/setup.bash
cd /workspace/work/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 launch ee616_evaluation pair_gate.launch.py \
  profile:=straight run_id:=run_01 \
  output_dir:=/workspace/work/ros2_ws/results/my_pair_trial
```

Use a new output directory for exploration.

Part IV

Configuration and Safe Modification

Chapter 16

Configuration hierarchy

16.1 Where values live

Location	Responsibility
<code>src/ee616_simulation/scenarios/*.yaml</code>	Static camera-development scenes.
<code>src/ee616_perception/config/*.yaml</code>	Detector, geometry, checkpoint, and calibration parameters.
<code>src/ee616_control/config/pair_control.yaml</code>	Desired gap, control bounds, gains, timeouts, and control rate.
<code>src/ee616_evaluation/launch/*.launch.py</code>	Experiment composition, run arguments, duration, namespaces, and output paths.
<code>experiments/*/experiment.yaml</code>	Frozen protocol, acceptance criteria, and scope.
<code>scripts/run*_gate.sh</code>	Repetition matrix, stage ordering, resource monitor, summary, and plots.

16.2 Precedence

ROS parameters supplied later in a launch parameter list override earlier YAML values. For example, the YOLO YAML defaults to `/smoke/camera/image`, but a pair launch overrides it with `/follower_1/camera/image`.

Chapter 17

Changing a scenario

17.1 Static scenario schema

The base scenario defines seed, floor, lighting, camera, target, and obstacles. A derived YAML uses one `extends:base.yaml` level. The loader prevents inheritance outside the scenario directory.

```
extends: base.yaml
name: my_test_scene
description: One sentence describing the scene.
variable_under_test: shelf distance
expected_visibility: visible

camera:
  pose: [-7.0, 0.0, 0.65, 0.0, 0.0, 0.0]

target:
  pose: [-3.5, 0.0, 0.6, 0.0, 0.0, 0.0]

obstacles:
  - name: left_shelf
    pose: [-2.5, 2.0, 1.0, 0.0, 0.0, 0.0]
    size: [8.0, 0.8, 2.0]
    color: [0.18, 0.32, 0.48, 1.0]
```

17.2 Pose fields

Gazebo pose lists are $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$. Linear units are metres and angular units are radians. Object size lists are $[x, y, z]$.

17.3 Safe scenario-change procedure

1. Copy the closest YAML and give it a new lowercase name.
2. Change one primary condition.
3. Describe the variable and expected visibility.
4. launch it interactively;

5. run the scenario validator;
6. declare metrics and thresholds before performance collection; and
7. write to a new result directory.

Chapter 18

Changing perception parameters

18.1 Confidence

Increasing confidence may reduce false positives and increase missed detections. Decreasing it may increase recall and false positives. Do not tune confidence on final test scenes.

18.2 IoU threshold

The intersection-over-union threshold affects non-maximum suppression. It is not a direct accuracy threshold. Change it only with a validation protocol.

18.3 Inference size

A smaller `image_size` can reduce computation but may harm small-target detection. A larger size can increase memory and latency. The current 640-pixel setting passed the 4 GiB VRAM constraint.

18.4 Device

`device:"0"` selects the first CUDA GPU. A CPU experiment must be recorded as a different hardware condition, not silently compared with GPU evidence.

18.5 Range calibration

Scale and offset must be fit only on training or validation data. Re-fitting them on a sealed test scene leaks final information into the model-selection process.

Chapter 19

Changing control parameters

19.1 Desired gap

Changing `desired_range_m` changes the formation objective and camera target size. It may invalidate both controller and detector assumptions.

19.2 Gains

Higher range gain corrects spacing faster but can increase overshoot and command saturation. Higher bearing gain turns more strongly but can produce oscillation or target loss. Rate gains add damping or anticipation but depend on EKF quality.

19.3 Timeouts

`track_timeout_s` must be smaller than `predict_timeout_s`. A longer prediction interval tolerates missing frames but lets the robot move longer without visual correction. A shorter interval stops earlier but can overreact to isolated dropped frames.

19.4 Bounds

Command bounds are part of the experiment definition. Raising them changes both dynamics and risk. Do not change bounds merely to make an existing gate pass.

19.5 Control rate

The accepted controller runs at 20 Hz. This is faster than the requested 15 Hz camera rate, so some controller cycles necessarily use the latest EKF prediction rather than a new image.

Chapter 20

Creating a new experiment correctly

20.1 Do not edit frozen evidence

Create a new experiment ID, configuration, result directory, and decision-log entry. Keep the accepted Gate 1–6 evidence unchanged.

20.2 Minimum protocol

Before executing, record:

1. objective and hypothesis;
2. code and configuration revision;
3. independent and controlled variables;
4. scenarios and repetitions;
5. acceptance criteria;
6. metrics and calculation method;
7. ground-truth boundary;
8. hardware and software condition;
9. failure-handling rule; and
10. output filenames and checksum method.

20.3 One-variable example

To study inference size, keep the checkpoint, confidence, route, controller, followers, scenes, and seeds fixed. Compare 416, 512, and 640 pixels. Measure detection validity, range and bearing error, inference p95, end-to-end p95, real-time factor, RAM, and VRAM. Do not call the fastest setting “best” unless the accuracy requirements also pass.

20.4 When optimization is justified

Gate 6 already met its frozen timing and memory limits. Optimization is therefore an optional research extension, not a repair required for the accepted project. A defensible extension would study accuracy–latency tradeoffs or adaptive detector scheduling under measured contention.

Part V

Metrics, Evidence, and Interpretation

Chapter 21

Measurement metrics

21.1 Range RMSE

For N synchronized valid samples,

$$\text{RMSE}_r = \sqrt{\frac{1}{N} \sum_{k=1}^N (\hat{r}_k - r_k)^2}. \quad (21.1)$$

RMSE emphasizes large errors because errors are squared. Always report the scenes, visibility rule, and valid sample count.

21.2 Bearing MAE

$$\text{MAE}_\beta = \frac{1}{N} \sum_{k=1}^N \left| \text{wrap}(\hat{\beta}_k - \beta_k) \right|. \quad (21.2)$$

Always state degrees or radians.

21.3 Precision and recall

Precision is $TP/(TP + FP)$. **Recall** is $TP/(TP + FN)$. The project also used localization-aware counting, so a detected box can be considered mislocalized rather than correct.

21.4 Valid measurement rate

This is the number of valid measurements divided by the number of full-visible evaluation samples. It is not the same as recall unless validity and ground-truth labeling use the same conditions.

Chapter 22

Formation and recovery metrics

22.1 Spacing RMSE

With desired range r_d ,

$$\text{RMSE}_{\text{spacing}} = \sqrt{\frac{1}{N} \sum_{k=1}^N (r_k - r_d)^2}. \quad (22.1)$$

The Gate 3–6 evaluator uses truth-derived range only for evaluation. The controller does not receive it.

22.2 TRACK fraction

TRACK fraction is the evaluated samples in TRACK divided by all evaluated samples. A system can have low spacing error while spending too much time without current visual measurements, so both metrics matter.

22.3 Reacquisition time

Reacquisition begins when a declared occlusion ends and finishes at the first later TRACK sample. The condition and sampling period must remain fixed across comparisons.

22.4 Collision sample

The evaluator flags a sample when declared center distance is below 0.65 m. Zero flagged samples means only that this rule was not triggered in recorded samples. It is not continuous collision proof.

22.5 Rearward error propagation

For adjacent followers, compare mean or worst RMSE under the same scenario. The nominal chain produced decreasing mean RMSE; the combined disturbance chain produced increasing mean RMSE. Therefore, the project does not claim general string stability.

Chapter 23

Timing and resource metrics

23.1 Latency stages

Gate 6 records observer-side wall-clock intervals:

- camera arrival to corresponding measurement arrival;
- valid measurement arrival to next controller command;
- controller command to post-proxy command;
- detector-reported inference time; and
- control estimate period.

These intervals occur on one laptop. They do not include a network link or physical actuator response.

23.2 Percentiles

The p95 value is the value at or below which approximately 95% of samples lie. It describes the upper tail better than a mean. Gate 6 also retained maxima, which were strongly affected by startup. The frozen acceptance rule used p95 values declared before execution.

23.3 Real-time factor

$$\text{RTF} = \frac{\text{simulated time elapsed}}{\text{wall time elapsed}}. \quad (23.1)$$

An RTF near 1.0 means the simulator advances approximately in real time.

23.4 CPU interpretation

CPU core-equivalent percentage can exceed 100% because several logical CPUs work concurrently. A value of 443% is roughly 4.43 fully occupied logical cores during the measured sample, not 443% of the whole 12-core capacity.

Chapter 24

Validation gates and frozen results

24.1 Gate 1: environment

The canonical container built the ROS workspace, passed namespace probes, and transported timestamped 640 by 480 RGB images. This established toolchain operation, not measurement accuracy.

24.2 Gate 2: camera measurement

The deterministic red-target baseline passed with 0.0355 m range RMSE and 0.120 degrees bearing MAE. YOLOv8n v1 and v2 failed their independent gates. Synchronized v3 passed with 100% valid full-visible measurements, 0.0323 m range RMSE, 0.1569 degrees bearing MAE, and 9.02 ms p95 inference time.

24.3 Gate 3: one pair

Six nominal runs passed. Worst spacing RMSE was 0.0558 m straight and 0.1005 m during gradual turns. The evaluator recorded zero collision samples.

24.4 Gate 4: incremental chain

Six two-follower trials passed before six three-follower trials were allowed. Three-follower worst RMSE was 0.1004, 0.0739, and 0.0523 m from front to rear. This nominal matrix did not show rearward amplification.

24.5 Gate 5: controlled disturbances

Fifteen pair trials passed before three combined-chain trials. Pair worst RMSE ranged from 0.0536 m for actuator bias to 0.1247 m for the sharp turn. Combined worst RMSE was 0.1007, 0.1242, and 0.1403 m. Long blackout trials produced the required prediction, stop, zero-command, and reacquisition sequence.

24.6 Gate 6: target-laptop timing

All three timing and corresponding behavior trials passed. Worst p95 values were 23.09 ms camera-to-measurement, 48.35 ms measurement-to-controller, 2.89 ms controller-to-post-proxy, 20.34 ms detector inference, and 50.0 ms control period. Minimum delivery was 95.4%, minimum RTF was 0.9996, peak container RAM was 3,584 MiB, and peak visible VRAM was 642 MiB.

24.7 Gate 7: freeze

Gate 7 records source, configurations, experiment protocols, evidence, manual, walkthrough video, and SHA-256 checksums. The verifier also checks that retained failed summaries still say **fail** and accepted summaries still say **pass**.

Chapter 25

Limitations and threats to validity

25.1 Simulation-to-reality gap

Rendered images, friction, dynamics, lighting, odometry, and timing differ from physical robots. Simulation results motivate later experiments but cannot establish physical performance.

25.2 Target assumption

The system requires a known 0.90 m target height and distinctive target appearance. A target-free predecessor detector would be a different research problem.

25.3 Occlusion model

The full-frame blackout removes every pixel, while a real occluder produces structured partial visibility. The existing partial-occlusion static template was not the Gate 5 blackout mechanism.

25.4 Actuator model

The command bias is applied after the controller but before Gazebo's command input. It does not model motor current limits, gearbox backlash, wheel slip, encoder bias, or traction loss.

25.5 Chain length and routes

The accepted dynamic chain contains at most three followers and a small set of deterministic routes. No general string-stability theorem or broad warehouse-navigation result follows.

25.6 Timing instrumentation

Observer callbacks add some load and measure arrival times rather than internal hardware timestamps for every stage. Startup outliers are retained, but p95 is the frozen criterion.

25.7 Known implementation debt

The follower estimate message currently sets its diagnostic `frame_id` to `follower_1/camera` even when the same node is launched under later follower namespaces. The numerical estimate is namespaced correctly and the accepted evaluator does not use this string for transforms, but a future revision should derive the frame ID from the namespace and revalidate it. Frozen evidence is not silently changed during Gate 7.

Part VI

Troubleshooting and Extension

Chapter 26

Troubleshooting

26.1 Blank host terminal or missing ROS command

The host intentionally does not provide the canonical ROS environment. Open the Dev Container or use `makecontainer-shell/makegpu-shell`. Do not source a nonexistent host Jazzy setup file.

26.2 Docker permission denied

Either use the approved `sudodocker` form or review Docker group membership. Do not change system permissions through project code.

26.3 Gazebo GUI does not open

Start from the graphical desktop session, confirm `DISPLAY` and `XAUTHORITY`, run `infrastructure/dev/prepare_xauthority.sh`, and separate display failure from headless simulator failure with `makeenvironment-check-gpu`.

26.4 CUDA or GPU not visible

Run `nvidia-smi` on the host and inside `makegpu-shell`. Confirm the NVIDIA Container Toolkit and GPU Compose profile. Do not silently fall back to CPU and compare the result with GPU timing evidence.

26.5 No camera frames

Check Gazebo topic listing, bridge process, ROS topic type, subscription count, timestamps, and rate. Confirm the generated world uses the expected topic and that the ROS workspace was rebuilt after source changes.

26.6 No detections

Check checkpoint path, CUDA device, RGB/BGR contract, confidence, target class, image topic, target visibility, and diagnostic boxes. Do not lower confidence until the image and label contract is verified.

26.7 Immediate `SAFE_STOP`

Inspect measurement validity, message age, ROS simulation time, camera rate, and detector startup. The accepted disturbance protocol includes eight seconds of stationary initialization because an earlier run exposed a startup race.

26.8 A gate script fails

Read the per-run summary before rerunning. Determine whether the failure is scientific, configuration-related, or evaluator-related. Preserve a meaningful failed result. Never lower a frozen threshold simply to make the summary pass.

Chapter 27

Reasonable future directions

27.1 Resource-aware perception

Study detector image size, frequency, scheduling, and number of followers while measuring accuracy, deadline compliance, RTF, RAM, and VRAM. This direction builds directly on Gate 6.

27.2 Physical occlusion scenes

Replace full-frame blackout with moving occluders, partial visibility, clutter, and lighting variation. Define detection and reacquisition conditions before testing.

27.3 Leader autonomy

A future leader may use 2-D LiDAR, wheel odometry, an IMU, and Nav2 with a grid-based planner such as Smac 2D A*. This would support leader route navigation. It should remain separate from the follower formation experiment so navigation failure is not confused with follower failure.

27.4 Robot assignment

Fleet assignment is a higher-level scheduling problem. It can assign followers to missions or leaders based on availability, payload, battery, and route cost. It is not required to validate the current pair or chain.

27.5 Physical robot validation

A physical stage would require calibrated cameras, encoder characterization, time synchronization, independent obstacle/person protection, controlled test space, emergency stop, risk review, and conservative speed. Simulation results alone do not authorize it.

Appendix A

Command reference

Command	Purpose
<code>makecontainer-build</code>	Build the canonical image.
<code>makeenvironment-check-gpu</code>	Revalidate ROS, Gazebo, tests, camera, and GPU profile.
<code>makegazebo-gui</code>	Open the learning playground.
<code>makescenario-guiSCENARIO=name</code>	Open one controlled template.
<code>makescenario-validate</code>	Validate all five static templates.
<code>makeaws-warehouse-gui</code>	Open the optional AWS world.
<code>makeaws-benchmark</code>	Run its fixed resource benchmark.
<code>makepair-gate</code>	Run six nominal pair trials.
<code>makechain-gate</code>	Run staged two- and three-follower trials.
<code>makedisturbance-gate</code>	Run pair-first controlled disturbances.
<code>maketiming-gate</code>	Run three timing/resource trials.
<code>python3work/gate7_package/ verify_frozen_evidence.py</code>	Verify the Gate 7 frozen package.

Appendix B

Topic and message reference

Topic pattern	Message	Meaning
/leader/cmd_vel	Twist	Leader motion command.
/leader/odom	Odometry	Leader local odometry.
/leader/route/state	String	Route phase for evaluation.
/follower_i/camera/image	Image	Raw simulated RGB camera.
/follower_i/camera/disturbed	Image	Gate 5 post-proxy camera.
/follower_i/measurement/ relative	Vector3Stamped	Range, bearing, and validity.
/follower_i/measurement/ inference_ms	Float32	Detector inference duration.
/follower_i/measurement/ detection_box	Float32MultiArray	Confidence and box diagnostics.
/follower_i/odom	Odometry	Follower local motion input.
/follower_i/controller_cmd_ vel	Twist	Pre-disturbance command in Gate 5.
/follower_i/cmd_vel	Twist	Command entering Gazebo.
/follower_i/control/state	String	Supervisor state.
/follower_i/control/estimate	Vector3Stamped	EKF range, bearing, and age.
/experiment/event	String	Declared disturbance label.

Appendix C

Accepted control parameters

Parameter	Value	Unit or meaning
desired_range_m	1.50	m
max_linear_mps	0.60	m/s
max_angular_rps	0.80	rad/s
track_timeout_s	0.20	s
predict_timeout_s	0.75	s
range_gain	0.85	linear proportional gain
range_rate_gain	0.85	range-rate gain
bearing_gain	1.80	angular proportional gain
bearing_rate_gain	0.20	bearing-rate gain
predict_speed_scale	0.55	multiplier in PREDICT
control_rate_hz	20.0	Hz

Appendix D

Glossary

Acceptance criterion

A condition declared before execution that determines whether an experiment passes.

Bearing

Horizontal angular displacement of the target from camera forward.

Bounding box

Rectangle returned by the detector around an image object.

Closed loop

System in which measured output influences later control input.

Covariance

Matrix representation of estimated uncertainty and cross-correlation.

Differential drive

Two-wheel drive model controlled by linear and angular velocity.

EKF

Extended Kalman filter; a nonlinear predict/update estimator using local linearization.

Evaluation-only truth

Simulator truth used only to calculate metrics after or alongside operation.

Field of view

Angular span visible to the camera.

Formation control

Control that maintains a desired relative arrangement between robots.

Ground truth

Reference state treated as the evaluator's best available true value.

Inference

Execution of a trained model on an input to obtain predictions.

Latency

Time between declared start and end events in a processing path.

MAE

Mean absolute error.

Namespace

ROS name prefix that isolates one robot's nodes and topics.

Odometry

Locally estimated robot motion or pose, often derived from wheels.

Open loop

Test in which measured output does not control the tested motion.

p95

Ninety-fifth percentile of a measured distribution.

Predecessor

Robot immediately ahead of a follower.

Precision

Fraction of reported detections that are correct.

Recall

Fraction of real targets that are correctly detected.

Real-time factor

Simulated elapsed time divided by wall-clock elapsed time.

RMSE

Root mean square error.

ROS topic

Named publish/subscribe channel for streaming messages.

Sealed test scene

Final evaluation scene excluded from training, validation, threshold selection, and calibration.

String stability

Property concerning disturbance or error propagation along a vehicle chain; not established by this project.

Supervisor

Logic that selects operating mode from validity and age conditions.

Validation data

Data used to choose model settings without using final test data.

YOLOv8n

Nano-sized YOLOv8 object detector used for the predecessor target.

Appendix E

Oral defense study questions

1. Why is one leader–follower pair the minimum scientific unit?
2. What information is available to a follower, and what information is forbidden?
3. Derive focal length, bearing, forward depth, and planar range from the detected box.
4. What are the four EKF states, and how does follower motion enter prediction?
5. Why does PREDICT exist between TRACK and SAFE_STOP?
6. Why is a full-frame blackout not the same as a physical occlusion?
7. Why did v1 and v2 YOLO detectors remain in the evidence record?
8. Why can nominal rearward error decrease while disturbed rearward error increases?
9. What does zero collision samples mean, and what does it not mean?
10. What does an RTF of 0.9996 tell you?
11. Why can CPU core-equivalent percentage exceed 100%?
12. Which result best supports feasibility on the target laptop?
13. What would have to change before a physical robot test?
14. What would you change if the professor requests a narrower project?

Appendix F

Frozen evidence locations

Evidence	Location
Toy pair summary	work/toy_simulation/results/summary.json
Toy warehouse summary	work/toy_simulation/warehouse_results/warehouse_multi_summary.json
Environment gate	work/ros2_ws/results/environment_gate/summary.json
Camera baseline	work/ros2_ws/results/camera_measurement_gate/summary.json
YOLO v1/v2 failures	work/ros2_ws/results/yolo_measurement_gate*/summary.json
YOLO v3 pass	work/ros2_ws/results/yolo_measurement_gate_v3/summary.json
Pair gate	work/ros2_ws/results/pair_gate/summary.json
Chain gate	work/ros2_ws/results/chain_gate/summary.json
Disturbance gate	work/ros2_ws/results/disturbance_gate/summary.json
Timing gate	work/ros2_ws/results/timing_gate/summary.json
Final evidence index	docs/FINAL_EVIDENCE_INDEX.md
Frozen checksums	work/gate7_package/FROZEN_SHA256SUMS

Bibliography

- [1] S. Knorn, A. Donaire, J. C. Agueero, and R. H. Middleton, “Decentralized 2-D Control of Vehicular Platoons under Limited Visual Feedback,” 2017. <https://arxiv.org/abs/1702.03097>.
- [2] “Adaptive Image-Based Leader-Follower Formation Control of Mobile Robots With Visibility Constraints,” *IEEE Transactions on Industrial Electronics*, 2020. <https://doi.org/10.1109/TIE.2020.2994861>.
- [3] “A Leader-Follower Formation Control of Mobile Robots by Position-Based Visual Servo Method Using Fisheye Camera,” 2023. <https://doi.org/10.1186/s40648-023-00268-6>.
- [4] “A Concurrent Learning Approach to Monocular Vision Range Regulation of Leader/Follower Systems,” 2024. <https://doi.org/10.1007/s10514-024-10178-0>.
- [5] “Leader-Follower Mobile Robot Using Visual Marker,” 2025. <https://doi.org/10.13067/JKIECS.2025.20.1.213>.
- [6] “System Integration and Field Testing of Low-Cost Autonomous Platooning Platforms,” 2025. <https://doi.org/10.1002/rob.70067>.
- [7] Open Robotics, “ROS 2 Jazzy Documentation.” <https://docs.ros.org/en/jazzy/>.
- [8] Open Source Robotics Foundation, “Gazebo Harmonic Documentation.” <https://gazebo.sim.org/docs/harmonic/>.
- [9] Ultralytics, “Ultralytics YOLO Documentation.” <https://docs.ultralytics.com/>.

Index

- acceptance criterion, 34
- actuator bias, 19
- AWS warehouse, 25
- Bearing, 6
- camera failure, 45
- camera model, 10
- chain gate, 26
- claim boundary, 4
- closed loop, 6
- closed-loop, 6
- colcon, 24
- collision sample, 38
- command saturation, 17
- confidence threshold, 11
- configuration, 29
- control gain, 33
- control rate, 33
- coordinate frame, 6
- covariance, 15
- CPU core-equivalent, 39
- CUDA device, 32
- desired gap, 33
- detection failure, 45
- determinism, 6
- Dev Container, 22
- differential drive, 10
- display error, 45
- disturbance, 19
- disturbance gate, 26
- Docker, 21
- Docker permission, 45
- EKF, *see* extended Kalman filter
- environment gate, 26
- evidence freeze, 40
- experiment protocol, 34
- experiment runner, 26
- extended Kalman filter, 15
- failed experiments, 11
- field of view, 13
- focal length, 13
- formation control, 17
- future work, 47
- Gate 1, 40
- Gate 2, 40
- Gate 3, 40
- Gate 4, 40
- Gate 5, 40
- Gate 6, 40
- Gate 7, 40
- Gazebo, 10
 - GUI, 25
 - Harmonic, 21
- GPU, 21
- Ground truth, 4
- gz command, 24
- independent variable, 34
- inference size, 32
- innovation, 15
- installation, 22
- intersection over union, 32
- latency, 39
- leader autonomy, 47
- leader route, 19
- leader–follower control, 2
- MAE, 37
- monocular geometry, 13
- namespace, 8
- network timing, 42
- NVIDIA Container Toolkit, 22
- occlusion, 19
- open loop, 6
- open-loop, 6
- optimization, 34

p95, 39
pair gate, 26
parameter precedence, 29
percentile, 39
perception parameter, 32
physical validation, 47
pose, 6
pose fields, 30
Precision, 37
precision, 37
predecessor following, 8
predecessor-following topology, 2
PREDICT, 17
production readiness, 4
project contribution, 2

Range, 6
range calibration, 13
reacquisition time, 38
real-time factor, 39
rearward error, 38
Recall, 37
recall, 37
reproducibility, 30
resource-aware perception, 47
RGB blackout, 19
RGB/BGR contract, 11
RMSE, 37
robot assignment, 47
ROS 2, 8
 Jazzy, 21
ROS parameter, 29
ros2 command, 24

SAFE STOP, 17
safety boundary, 4
scenario
 custom, 30
scenario template, 25
shutdown, 24
simulation-to-reality gap, 42
spacing RMSE, 38

technical debt, 42
threats to validity, 42
timeout, 33
timing gate, 26
topic, 8
TRACK, 17
TRACK fraction, 38
transform, 6
troubleshooting, 45

Ubuntu Noble, 21

valid measurement rate, 37
validation gate, 40
visual target, 10
VRAM, 39
VS Code, 22

warehouse material transport, 2
Wheel odometry, 6
wheel slip, 42

YAML, 30
YOLOv8n, 11